

4.3 Asymptotic Analysis

In algorithm analysis, we focus on the growth rate of the running time as a function of the input size n , taking a “big-picture” approach. For example, it is often enough just to know that the running time of an algorithm *grows proportionally to n* .

We analyze algorithms using a mathematical notation for functions that disregards constant factors. Namely, we characterize the running times of algorithms by using functions that map the size of the input, n , to values that correspond to the main factor that determines the growth rate in terms of n . This approach reflects that each basic step in a pseudocode description or a high-level language implementation may correspond to a small number of primitive operations. Thus, we can perform an analysis of an algorithm by estimating the number of primitive operations executed up to a constant factor, rather than getting bogged down in language-specific or hardware-specific analysis of the exact number of operations that execute on the computer.

4.3.1 The “Big-Oh” Notation

Let $f(n)$ and $g(n)$ be functions mapping positive integers to positive real numbers. We say that $f(n)$ is $O(g(n))$ if there is a real constant $c > 0$ and an integer constant $n_0 \geq 1$ such that

$$f(n) \leq c \cdot g(n), \quad \text{for } n \geq n_0.$$

This definition is often referred to as the “big-Oh” notation, for it is sometimes pronounced as “ $f(n)$ is *big-Oh* of $g(n)$.” Figure 4.5 illustrates the general definition.

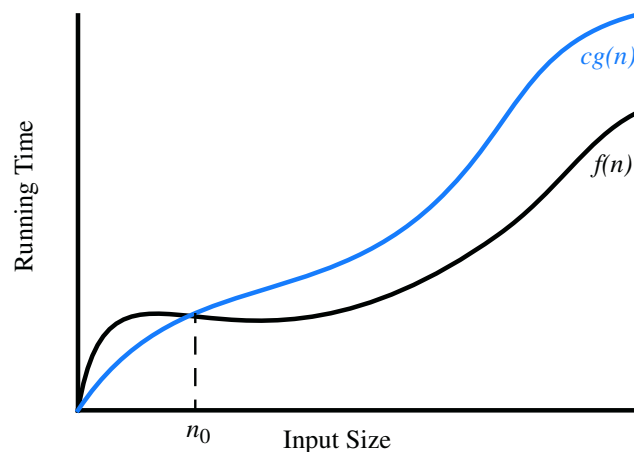


Figure 4.5: Illustrating the “big-Oh” notation. The function $f(n)$ is $O(g(n))$, since $f(n) \leq c \cdot g(n)$ when $n \geq n_0$.

Example 4.6: The function $8n + 5$ is $O(n)$.

Justification: By the big-Oh definition, we need to find a real constant $c > 0$ and an integer constant $n_0 \geq 1$ such that $8n + 5 \leq cn$ for every integer $n \geq n_0$. It is easy to see that a possible choice is $c = 9$ and $n_0 = 5$. Indeed, this is one of infinitely many choices available because there is a trade-off between c and n_0 . For example, we could rely on constants $c = 13$ and $n_0 = 1$. ■

The big-Oh notation allows us to say that a function $f(n)$ is “less than or equal to” another function $g(n)$ up to a constant factor and in the *asymptotic* sense as n grows toward infinity. This ability comes from the fact that the definition uses “ $\leq$ ” to compare $f(n)$ to a $g(n)$ times a constant, c , for the asymptotic cases when $n \geq n_0$. However, it is considered poor taste to say “ $f(n) \leq O(g(n))$,” since the big-Oh already denotes the “less-than-or-equal-to” concept. Likewise, although common, it is not fully correct to say “ $f(n) = O(g(n))$,” with the usual understanding of the “ $=$ ” relation, because there is no way to make sense of the symmetric statement, “ $O(g(n)) = f(n)$.” It is best to say, “ $f(n)$ is $O(g(n))$.”

Alternatively, we can say “ $f(n)$ is *order of* $g(n)$.” For the more mathematically inclined, it is also correct to say, “ $f(n) \in O(g(n))$,” for the big-Oh notation, technically speaking, denotes a whole collection of functions. In this book, we will stick to presenting big-Oh statements as “ $f(n)$ is $O(g(n))$.” Even with this interpretation, there is considerable freedom in how we can use arithmetic operations with the big-Oh notation, and with this freedom comes a certain amount of responsibility.

Some Properties of the Big-Oh Notation

The big-Oh notation allows us to ignore constant factors and lower-order terms and focus on the main components of a function that affect its growth.

Example 4.7: $5n^4 + 3n^3 + 2n^2 + 4n + 1$ is $O(n^4)$.

Justification: Note that $5n^4 + 3n^3 + 2n^2 + 4n + 1 \leq (5 + 3 + 2 + 4 + 1)n^4 = cn^4$, for $c = 15$, when $n \geq n_0 = 1$. ■

In fact, we can characterize the growth rate of any polynomial function.

Proposition 4.8: If $f(n)$ is a polynomial of degree d , that is,

$$f(n) = a_0 + a_1n + \cdots + a_dn^d,$$

and $a_d > 0$, then $f(n)$ is $O(n^d)$.

Justification: Note that, for $n \geq 1$, we have $1 \leq n \leq n^2 \leq \cdots \leq n^d$; hence,

$$a_0 + a_1n + a_2n^2 + \cdots + a_dn^d \leq (|a_0| + |a_1| + |a_2| + \cdots + |a_d|)n^d.$$

We show that $f(n)$ is $O(n^d)$ by defining $c = |a_0| + |a_1| + \cdots + |a_d|$ and $n_0 = 1$. ■

Thus, the highest-degree term in a polynomial is the term that determines the asymptotic growth rate of that polynomial. We consider some additional properties of the big-Oh notation in the exercises. Let us consider some further examples here, focusing on combinations of the seven fundamental functions used in algorithm design. We rely on the mathematical fact that $\log n \leq n$ for $n \geq 1$.

Example 4.9: $5n^2 + 3n \log n + 2n + 5$ is $O(n^2)$.

Justification: $5n^2 + 3n \log n + 2n + 5 \leq (5 + 3 + 2 + 5)n^2 = cn^2$, for $c = 15$, when $n \geq n_0 = 1$. ■

Example 4.10: $20n^3 + 10n \log n + 5$ is $O(n^3)$.

Justification: $20n^3 + 10n \log n + 5 \leq 35n^3$, for $n \geq 1$. ■

Example 4.11: $3 \log n + 2$ is $O(\log n)$.

Justification: $3 \log n + 2 \leq 5 \log n$, for $n \geq 2$. Note that $\log n$ is zero for $n = 1$. That is why we use $n \geq n_0 = 2$ in this case. ■

Example 4.12: 2^{n+2} is $O(2^n)$.

Justification: $2^{n+2} = 2^n \cdot 2^2 = 4 \cdot 2^n$; hence, we can take $c = 4$ and $n_0 = 1$ in this case. ■

Example 4.13: $2n + 100 \log n$ is $O(n)$.

Justification: $2n + 100 \log n \leq 102n$, for $n \geq n_0 = 1$; hence, we can take $c = 102$ in this case. ■

Characterizing Functions in Simplest Terms

In general, we should use the big-Oh notation to characterize a function as closely as possible. While it is true that the function $f(n) = 4n^3 + 3n^2$ is $O(n^5)$ or even $O(n^4)$, it is more accurate to say that $f(n)$ is $O(n^3)$. Consider, by way of analogy, a scenario where a hungry traveler driving along a long country road happens upon a local farmer walking home from a market. If the traveler asks the farmer how much longer he must drive before he can find some food, it may be truthful for the farmer to say, “certainly no longer than 12 hours,” but it is much more accurate (and helpful) for him to say, “you can find a market just a few minutes drive up this road.” Thus, even with the big-Oh notation, we should strive as much as possible to tell the whole truth.

It is also considered poor taste to include constant factors and lower-order terms in the big-Oh notation. For example, it is not fashionable to say that the function $2n^2$ is $O(4n^2 + 6n \log n)$, although this is completely correct. We should strive instead to describe the function in the big-Oh in *simplest terms*.

The seven functions listed in Section 4.2 are the most common functions used in conjunction with the big-Oh notation to characterize the running times and space usage of algorithms. Indeed, we typically use the names of these functions to refer to the running times of the algorithms they characterize. So, for example, we would say that an algorithm that runs in worst-case time $4n^2 + n \log n$ is a **quadratic-time** algorithm, since it runs in $O(n^2)$ time. Likewise, an algorithm running in time at most $5n + 20 \log n + 4$ would be called a **linear-time** algorithm.

Big-Omega

Just as the big-Oh notation provides an asymptotic way of saying that a function is “less than or equal to” another function, the following notations provide an asymptotic way of saying that a function grows at a rate that is “greater than or equal to” that of another.

Let $f(n)$ and $g(n)$ be functions mapping positive integers to positive real numbers. We say that $f(n)$ is $\Omega(g(n))$, pronounced “ $f(n)$ is big-Omega of $g(n)$,” if $g(n)$ is $O(f(n))$, that is, there is a real constant $c > 0$ and an integer constant $n_0 \geq 1$ such that

$$f(n) \geq cg(n), \quad \text{for } n \geq n_0.$$

This definition allows us to say asymptotically that one function is greater than or equal to another, up to a constant factor.

Example 4.14: $3n \log n - 2n$ is $\Omega(n \log n)$.

Justification: $3n \log n - 2n = n \log n + 2n(\log n - 1) \geq n \log n$ for $n \geq 2$; hence, we can take $c = 1$ and $n_0 = 2$ in this case. ■

Big-Theta

In addition, there is a notation that allows us to say that two functions grow at the same rate, up to constant factors. We say that $f(n)$ is $\Theta(g(n))$, pronounced “ $f(n)$ is big-Theta of $g(n)$,” if $f(n)$ is $O(g(n))$ and $f(n)$ is $\Omega(g(n))$, that is, there are real constants $c' > 0$ and $c'' > 0$, and an integer constant $n_0 \geq 1$ such that

$$c'g(n) \leq f(n) \leq c''g(n), \quad \text{for } n \geq n_0.$$

Example 4.15: $3n \log n + 4n + 5 \log n$ is $\Theta(n \log n)$.

Justification: $3n \log n \leq 3n \log n + 4n + 5 \log n \leq (3 + 4 + 5)n \log n$ for $n \geq 2$. ■

4.3.2 Comparative Analysis

The big-Oh notation is widely used to characterize running times and space bounds in terms of some parameter n , which is defined as a chosen measure of the “size” of the problem. Suppose two algorithms solving the same problem are available: an algorithm A , which has a running time of $O(n)$, and an algorithm B , which has a running time of $O(n^2)$. Which algorithm is better? We know that n is $O(n^2)$, which implies that algorithm A is **asymptotically better** than algorithm B , although for a small value of n , B may have a lower running time than A .

We can use the big-Oh notation to order classes of functions by asymptotic growth rate. Our seven functions are ordered by increasing growth rate in the following sequence, such that $f(n)$ is $O(g(n))$ if function $f(n)$ precedes function $g(n)$:

$$1, \quad \log n, \quad n, \quad n \log n, \quad n^2, \quad n^3, \quad 2^n.$$

We illustrate the growth rates of the seven functions in Table 4.3. (See also Figure 4.4 from Section 4.2.1.)

n	$\log n$	n	$n \log n$	n^2	n^3	2^n
8	3	8	24	64	512	256
16	4	16	64	256	4,096	65,536
32	5	32	160	1,024	32,768	4,294,967,296
64	6	64	384	4,096	262,144	1.84×10^{19}
128	7	128	896	16,384	2,097,152	3.40×10^{38}
256	8	256	2,048	65,536	16,777,216	1.15×10^{77}
512	9	512	4,608	262,144	134,217,728	1.34×10^{154}

Table 4.3: Selected values of fundamental functions in algorithm analysis.

We further illustrate the importance of the asymptotic viewpoint in Table 4.4. This table explores the maximum size allowed for an input instance that is processed by an algorithm in 1 second, 1 minute, and 1 hour. It shows the importance of good algorithm design, because an asymptotically slow algorithm is beaten in the long run by an asymptotically faster algorithm, even if the constant factor for the asymptotically faster algorithm is worse.

Running Time (μs)	Maximum Problem Size (n)		
	1 second	1 minute	1 hour
$400n$	2,500	150,000	9,000,000
$2n^2$	707	5,477	42,426
2^n	19	25	31

Table 4.4: Maximum size of a problem that can be solved in 1 second, 1 minute, and 1 hour, for various running times measured in microseconds.

The importance of good algorithm design goes beyond just what can be solved effectively on a given computer, however. As shown in Table 4.5, even if we achieve a dramatic speedup in hardware, we still cannot overcome the handicap of an asymptotically slow algorithm. This table shows the new maximum problem size achievable for any fixed amount of time, assuming algorithms with the given running times are now run on a computer 256 times faster than the previous one.

Running Time	New Maximum Problem Size
$400n$	$256m$
$2n^2$	$16m$
2^n	$m + 8$

Table 4.5: Increase in the maximum size of a problem that can be solved in a fixed amount of time, by using a computer that is 256 times faster than the previous one. Each entry is a function of m , the previous maximum problem size.

Some Words of Caution

A few words of caution about asymptotic notation are in order at this point. First, note that the use of the big-Oh and related notations can be somewhat misleading should the constant factors they “hide” be very large. For example, while it is true that the function $10^{100}n$ is $O(n)$, if this is the running time of an algorithm being compared to one whose running time is $10n \log n$, we should prefer the $O(n \log n)$ -time algorithm, even though the linear-time algorithm is asymptotically faster. This preference is because the constant factor, 10^{100} , which is called “one googol,” is believed by many astronomers to be an upper bound on the number of atoms in the observable universe. So we are unlikely to ever have a real-world problem that has this number as its input size.

The observation above raises the issue of what constitutes a “fast” algorithm. Generally speaking, any algorithm running in $O(n \log n)$ time (with a reasonable constant factor) should be considered efficient. Even an $O(n^2)$ -time function may be fast enough in some contexts, that is, when n is small. But an algorithm whose running time is an exponential function, e.g., $O(2^n)$, should almost never be considered efficient.

Exponential Running Times

To see how fast the function 2^n grows, consider the famous story about the inventor of the game of chess. He asked only that his king pay him 1 grain of rice for the first square on the board, 2 grains for the second, 4 grains for the third, 8 for the fourth, and so on. The number of grains in the 64th square would be

$$2^{63} = 9,223,372,036,854,775,808,$$

which is about nine billion billions!

If we must draw a line between efficient and inefficient algorithms, therefore, it is natural to make this distinction be that between those algorithms running in polynomial time and those running in exponential time. That is, make the distinction between algorithms with a running time that is $O(n^c)$, for some constant $c > 1$, and those with a running time that is $O(b^n)$, for some constant $b > 1$. Like so many notions we have discussed in this section, this too should be taken with a “grain of salt,” for an algorithm running in $O(n^{100})$ time should probably not be considered “efficient.” Even so, the distinction between polynomial-time and exponential-time algorithms is considered a robust measure of tractability.

4.3.3 Examples of Algorithm Analysis

Now that we have the big-Oh notation for doing algorithm analysis, let us give some examples by characterizing the running time of some simple algorithms using this notation. Moreover, in keeping with our earlier promise, we will illustrate below how each of the seven functions given earlier in this chapter can be used to characterize the running time of an example algorithm.

Constant-Time Operations

All of the primitive operations, originally described on page 154, are assumed to run in constant time; formally, we say they run in $O(1)$ time. We wish to emphasize several important constant-time operations that involve arrays. Assume that variable A is an array of n elements. The expression $A.length$ in Java is evaluated in constant time, because arrays are represented internally with an explicit variable that records the length of the array. Another central behavior of arrays is that for any valid index j , the individual element, $A[j]$, can be accessed in constant time. This is because an array uses a consecutive block of memory. The j^{th} element can be found, not by iterating through the array one element at a time, but by validating the index, and using it as an offset from the beginning of the array in determining the appropriate memory address. Therefore, we say that the expression $A[j]$ is evaluated in $O(1)$ time for an array.

Finding the Maximum of an Array

As a classic example of an algorithm with a running time that grows proportional to n , we consider the goal of finding the largest element of an array. A typical strategy is to loop through elements of the array while maintaining as a variable the largest element seen thus far. Code Fragment 4.3 presents a method named `arrayMax` implementing this strategy.

```

1  /** Returns the maximum value of a nonempty array of numbers. */
2  public static double arrayMax(double[] data) {
3      int n = data.length;
4      double currentMax = data[0];           // assume first entry is biggest (for now)
5      for (int j=1; j < n; j++)              // consider all other entries
6          if (data[j] > currentMax)          // if data[j] is biggest thus far...
7              currentMax = data[j];         // record it as the current max
8      return currentMax;
9  }

```

Code Fragment 4.3: A method that returns the maximum value of an array.

Using the big-Oh notation, we can write the following mathematically precise statement on the running time of algorithm `arrayMax` for *any* computer.

Proposition 4.16: *The algorithm, `arrayMax`, for computing the maximum element of an array of n numbers, runs in $O(n)$ time.*

Justification: The initialization at lines 3 and 4 and the **return** statement at line 8 require only a constant number of primitive operations. Each iteration of the loop also requires only a constant number of primitive operations, and the loop executes $n - 1$ times. Therefore, we account for the number of primitive operations being $c' \cdot (n - 1) + c''$ for appropriate constants c' and c'' that reflect, respectively, the work performed inside and outside the loop body. Because each primitive operation runs in constant time, we have that the running time of algorithm `arrayMax` on an input of size n is at most $c' \cdot (n - 1) + c'' = c' \cdot n + (c'' - c') \leq c' \cdot n$ if we assume, without loss of generality, that $c'' \leq c'$. We conclude that the running time of algorithm `arrayMax` is $O(n)$. ■

Further Analysis of the Maximum-Finding Algorithm

A more interesting question about `arrayMax` is how many times we might update the current “biggest” value. In the worst case, if the data is given to us in increasing order, the biggest value is reassigned $n - 1$ times. But what if the input is given to us in random order, with all orders equally likely; what would be the expected number of times we update the biggest value in this case? To answer this question, note that we update the current biggest in an iteration of the loop only if the current element is bigger than all the elements that precede it. If the sequence is given to us in random order, the probability that the j^{th} element is the largest of the first j elements is $1/j$ (assuming uniqueness). Hence, the expected number of times we update the biggest (including initialization) is $H_n = \sum_{j=1}^n 1/j$, which is known as the n^{th} **Harmonic number**. It can be shown that H_n is $O(\log n)$. Therefore, the expected number of times the biggest value is updated by `arrayMax` on a randomly ordered sequence is $O(\log n)$.

Composing Long Strings

As our next example, we revisit the experimental study from Section 4.1, in which we examined two different implementations for composing a long string (see Code Fragment 4.2). Our first algorithm was based on repeated use of the string concatenation operator; for convenience, that method is also given in Code Fragment 4.4.

```

1  /** Uses repeated concatenation to compose a String with n copies of character c. */
2  public static String repeat1(char c, int n) {
3      String answer = "";
4      for (int j=0; j < n; j++)
5          answer += c;
6      return answer;
7  }
```

Code Fragment 4.4: Composing a string using repeated concatenation.

The most important aspect of this implementation is that strings in Java are *immutable* objects. Once created, an instance cannot be modified. The command, `answer += c`, is shorthand for `answer = (answer + c)`. This command does not cause a new character to be added to the existing `String` instance; instead it produces a new `String` with the desired sequence of characters, and then it reassigns the variable, `answer`, to refer to that new string.

In terms of efficiency, the problem with this interpretation is that the creation of a new string as a result of a concatenation, requires time that is proportional to the length of the resulting string. The first time through this loop, the result has length 1, the second time through the loop the result has length 2, and so on, until we reach the final string of length n . Therefore, the overall time taken by this algorithm is proportional to

$$1 + 2 + \cdots + n,$$

which we recognize as the familiar $O(n^2)$ summation from Proposition 4.3. Therefore, the total time complexity of the `repeat1` algorithm is $O(n^2)$.

We see this theoretical analysis reflected in the experimental results. The running time of a quadratic algorithm should theoretically quadruple if the size of the problem doubles, as $(2n)^2 = 4 \cdot n^2$. (We say “theoretically,” because this does not account for lower-order terms that are hidden by the asymptotic notation.) We see such an approximate fourfold increase in the running time of `repeat1` in Table 4.1 on page 152.

In contrast, the running times in that table for the `repeat2` algorithm, which uses Java’s `StringBuilder` class, demonstrate a trend of approximately *doubling* each time the problem size doubles. The `StringBuilder` class relies on an advanced technique with a worst-case running time of $O(n)$ for composing a string of length n ; we will later explore that technique as the focus of Section 7.2.1.

Three-Way Set Disjointness

Suppose we are given three sets, A , B , and C , stored in three different integer arrays. We will assume that no individual set contains duplicate values, but that there may be some numbers that are in two or three of the sets. The *three-way set disjointness* problem is to determine if the intersection of the three sets is empty, namely, that there is no element x such that $x \in A$, $x \in B$, and $x \in C$. A simple Java method to determine this property is given in Code Fragment 4.5.

```

1  /** Returns true if there is no element common to all three arrays. */
2  public static boolean disjoint1(int[] groupA, int[] groupB, int[] groupC) {
3      for (int a : groupA)
4          for (int b : groupB)
5              for (int c : groupC)
6                  if ((a == b) && (b == c))
7                      return false;           // we found a common value
8      return true;                           // if we reach this, sets are disjoint
9  }
```

Code Fragment 4.5: Algorithm `disjoint1` for testing three-way set disjointness.

This simple algorithm loops through each possible triple of values from the three sets to see if those values are equivalent. If each of the original sets has size n , then the worst-case running time of this method is $O(n^3)$.

We can improve upon the asymptotic performance with a simple observation. Once inside the body of the loop over B , if selected elements a and b do not match each other, it is a waste of time to iterate through all values of C looking for a matching triple. An improved solution to this problem, taking advantage of this observation, is presented in Code Fragment 4.6.

```

1  /** Returns true if there is no element common to all three arrays. */
2  public static boolean disjoint2(int[] groupA, int[] groupB, int[] groupC) {
3      for (int a : groupA)
4          for (int b : groupB)
5              if (a == b)                       // only check C when we find match from A and B
6                  for (int c : groupC)
7                      if (a == c)               // and thus b == c as well
8                          return false;         // we found a common value
9      return true;                             // if we reach this, sets are disjoint
10 }
```

Code Fragment 4.6: Algorithm `disjoint2` for testing three-way set disjointness.

In the improved version, it is not simply that we save time if we get lucky. We claim that the *worst-case* running time for `disjoint2` is $O(n^2)$. There are quadratically many pairs (a, b) to consider. However, if A and B are each sets of distinct

elements, there can be at most $O(n)$ such pairs with a equal to b . Therefore, the innermost loop, over C , executes at most n times.

To account for the overall running time, we examine the time spent executing each line of code. The management of the for loop over A requires $O(n)$ time. The management of the for loop over B accounts for a total of $O(n^2)$ time, since that loop is executed n different times. The test $a == b$ is evaluated $O(n^2)$ times. The rest of the time spent depends upon how many matching (a, b) pairs exist. As we have noted, there are at most n such pairs; therefore, the management of the loop over C and the commands within the body of that loop use at most $O(n^2)$ time. By our standard application of Proposition 4.8, the total time spent is $O(n^2)$.

Element Uniqueness

A problem that is closely related to the three-way set disjointness problem is the **element uniqueness problem**. In the former, we are given three sets and we presumed that there were no duplicates within a single set. In the element uniqueness problem, we are given an array with n elements and asked whether all elements of that collection are distinct from each other.

Our first solution to this problem uses a straightforward iterative algorithm. The `unique1` method, given in Code Fragment 4.7, solves the element uniqueness problem by looping through all distinct pairs of indices $j < k$, checking if any of those pairs refer to elements that are equivalent to each other. It does this using two nested for loops, such that the first iteration of the outer loop causes $n - 1$ iterations of the inner loop, the second iteration of the outer loop causes $n - 2$ iterations of the inner loop, and so on. Thus, the worst-case running time of this method is proportional to

$$(n - 1) + (n - 2) + \cdots + 2 + 1,$$

which we recognize as the familiar $O(n^2)$ summation from Proposition 4.3.

```

1  /** Returns true if there are no duplicate elements in the array. */
2  public static boolean unique1(int[] data) {
3      int n = data.length;
4      for (int j=0; j < n-1; j++)
5          for (int k=j+1; k < n; k++)
6              if (data[j] == data[k])
7                  return false;           // found duplicate pair
8      return true;                       // if we reach this, elements are unique
9  }
```

Code Fragment 4.7: Algorithm `unique1` for testing element uniqueness.

Using Sorting as a Problem-Solving Tool

An even better algorithm for the element uniqueness problem is based on using sorting as a problem-solving tool. In this case, by sorting the array of elements, we are guaranteed that any duplicate elements will be placed next to each other. Thus, to determine if there are any duplicates, all we need to do is perform a single pass over the sorted array, looking for *consecutive* duplicates.

A Java implementation of this algorithm is given in Code Fragment 4.8. (See Section 3.1.3 for discussion of the `java.util.Arrays` class.)

```

1  /** Returns true if there are no duplicate elements in the array. */
2  public static boolean unique2(int[] data) {
3      int n = data.length;
4      int[] temp = Arrays.copyOf(data, n);    // make copy of data
5      Arrays.sort(temp);                     // and sort the copy
6      for (int j=0; j < n-1; j++)
7          if (temp[j] == temp[j+1])          // check neighboring entries
8              return false;                 // found duplicate pair
9      return true;                           // if we reach this, elements are unique
10 }
```

Code Fragment 4.8: Algorithm `unique2` for testing element uniqueness.

Sorting algorithms will be the focus of Chapter 12. The best sorting algorithms (including those used by `Array.sort` in Java) guarantee a worst-case running time of $O(n \log n)$. Once the data is sorted, the subsequent loop runs in $O(n)$ time, and so the entire `unique2` algorithm runs in $O(n \log n)$ time. Exercise C-4.35 explores the use of sorting to solve the three-way set disjointness problem in $O(n \log n)$ time.

Prefix Averages

The next problem we consider is computing what are known as *prefix averages* of a sequence of numbers. Namely, given a sequence x consisting of n numbers, we want to compute a sequence a such that a_j is the average of elements $x_0, \dots, x_j$, for $j = 0, \dots, n-1$, that is,

$$a_j = \frac{\sum_{i=0}^j x_i}{j+1}.$$

Prefix averages have many applications in economics and statistics. For example, given the year-by-year returns of a mutual fund, ordered from recent to past, an investor will typically want to see the fund's average annual returns for the most recent year, the most recent three years, the most recent five years, and so on. Likewise, given a stream of daily Web usage logs, a website manager may wish to track average usage trends over various time periods. We present two implementations for computing prefix averages, yet with significantly different running times.

A Quadratic-Time Algorithm

Our first algorithm for computing prefix averages, denoted as `prefixAverage1`, is shown in Code Fragment 4.9. It computes each element a_j independently, using an inner loop to compute that partial sum.

```

1  /** Returns an array a such that, for all j, a[j] equals the average of x[0], ..., x[j]. */
2  public static double[] prefixAverage1(double[] x) {
3      int n = x.length;
4      double[] a = new double[n];           // filled with zeros by default
5      for (int j=0; j < n; j++) {
6          double total = 0;                 // begin computing x[0] + ... + x[j]
7          for (int i=0; i <= j; i++)
8              total += x[i];
9          a[j] = total / (j+1);             // record the average
10     }
11     return a;
12 }
```

Code Fragment 4.9: Algorithm `prefixAverage1`.

Let us analyze the `prefixAverage1` algorithm.

- The initialization of $n = x.length$ at line 3 and the eventual return of a reference to array `a` at line 11 both execute in $O(1)$ time.
- Creating and initializing the new array, `a`, at line 4 can be done with in $O(n)$ time, using a constant number of primitive operations per element.
- There are two nested **for** loops, which are controlled, respectively, by counters j and i . The body of the outer loop, controlled by counter j , is executed n times, for $j = 0, \dots, n-1$. Therefore, statements `total = 0` and `a[j] = total / (j+1)` are executed n times each. This implies that these two statements, plus the management of counter j in the loop, contribute a number of primitive operations proportional to n , that is, $O(n)$ time.
- The body of the inner loop, which is controlled by counter i , is executed $j+1$ times, depending on the current value of the outer loop counter j . Thus, statement `total += x[i]`, in the inner loop, is executed $1 + 2 + 3 + \dots + n$ times. By recalling Proposition 4.3, we know that $1 + 2 + 3 + \dots + n = n(n+1)/2$, which implies that the statement in the inner loop contributes $O(n^2)$ time. A similar argument can be done for the primitive operations associated with maintaining counter i , which also take $O(n^2)$ time.

The running time of implementation `prefixAverage1` is given by the sum of these terms. The first term is $O(1)$, the second and third terms are $O(n)$, and the fourth term is $O(n^2)$. By a simple application of Proposition 4.8, the running time of `prefixAverage1` is $O(n^2)$.

A Linear-Time Algorithm

An intermediate value in the computation of the prefix average is the *prefix sum* $x_0 + x_1 + \cdots + x_j$, denoted as *total* in our first implementation; this allows us to compute the prefix average $a[j] = \text{total} / (j + 1)$. In our first algorithm, the prefix sum is computed anew for each value of j . That contributed $O(j)$ time for each j , leading to the quadratic behavior.

For greater efficiency, we can maintain the current prefix sum dynamically, effectively computing $x_0 + x_1 + \cdots + x_j$ as $\text{total} + x_j$, where value *total* is equal to the sum $x_0 + x_1 + \cdots + x_{j-1}$, when computed by the previous pass of the loop over j . Code Fragment 4.10 provides a new implementation, denoted as `prefixAverage2`, using this approach.

```

1  /** Returns an array a such that, for all j, a[j] equals the average of x[0], ..., x[j]. */
2  public static double[] prefixAverage2(double[] x) {
3      int n = x.length;
4      double[] a = new double[n];           // filled with zeros by default
5      double total = 0;                     // compute prefix sum as x[0] + x[1] + ...
6      for (int j=0; j < n; j++) {
7          total += x[j];                     // update prefix sum to include x[j]
8          a[j] = total / (j+1);              // compute average based on current sum
9      }
10     return a;
11 }
```

Code Fragment 4.10: Algorithm `prefixAverage2`.

The analysis of the running time of algorithm `prefixAverage2` follows:

- Initializing variables *n* and *total* uses $O(1)$ time.
- Initializing the array *a* uses $O(n)$ time.
- There is a single **for** loop, which is controlled by counter j . The maintenance of that loop contributes a total of $O(n)$ time.
- The body of the loop is executed n times, for $j = 0, \dots, n-1$. Thus, statements `total += x[j]` and `a[j] = total / (j+1)` are executed n times each. Since each of these statements uses $O(1)$ time per iteration, their overall contribution is $O(n)$ time.
- The eventual return of a reference to array *A* uses $O(1)$ time.

The running time of algorithm `prefixAverage2` is given by the sum of the five terms. The first and last are $O(1)$ and the remaining three are $O(n)$. By a simple application of Proposition 4.8, the running time of `prefixAverage2` is $O(n)$, which is much better than the quadratic time of algorithm `prefixAverage1`.